

docker-compose.yml

El archivo **docker-compose.yml** es el encargado de definir y administrar toda la infraestructura del sistema **C5 Campus** mediante contenedores Docker, su principal objetivo es automatizar la creación, configuración y comunicación entre todos los microservicios que participan en el funcionamiento del botón de pánico y del sistema de monitoreo inteligente. Gracias a este archivo, todos los servicios pueden iniciarse con un solo comando, garantizando que trabajen de forma coordinada dentro de una misma red.

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
1  version: "3.9"
2  networks:
3    c5net:
4      driver: bridge
```

Se crea una red llamada **c5net** utilizando el controlador *bridge*, esta red permite que todos los contenedores del sistema puedan comunicarse entre sí utilizando sus nombres de servicio como si estuvieran conectados dentro de una misma red local, gracias a esto, servicios como Geolocation pueden comunicarse directamente con Priority sin necesidad de conocer direcciones IP específicas.

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
5
6  volumes:
7    postgres_master_data:
8    postgres_replica_data:
9    redis_data:
10   mosquito_data:
```

Los volúmenes permiten almacenar información de forma permanente fuera de los contenedores. De esta manera, si un contenedor es eliminado o reiniciado, los datos continúan disponibles. Se crean volúmenes para la base de datos principal, la réplica, Redis y Mosquitto

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
12  services:
13      mosquito:
14          image: eclipse-mosquitto:2.0
15          container_name: c5_mosquitto
16          ports:
17              - "1883:1883"
18              - "9001:9001"
19          volumes:
20              - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf
21              - mosquito_data:/mosquitto/data
22          networks:
23              - c5net
24          restart: unless-stopped
```

Mosquitto es el broker MQTT utilizado para la comunicación con dispositivos IoT y tótems de emergencia. El puerto 1883 permite la comunicación MQTT tradicional, mientras que el puerto 9001 habilita conexiones mediante WebSockets.

Se monta el archivo de configuración personalizado y un volumen para almacenar información persistente.

Indica que el servicio se reiniciará automáticamente en caso de falla, excepto si es detenido manualmente.

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
26  redis:
27      image: redis:7-alpine
28      container_name: c5_redis
29      ports:
30          - "6379:6379"
31      volumes:
32          - redis_data:/data
33      networks:
34          - c5net
35      restart: unless-stopped
```

Redis funciona como una base de datos en memoria utilizada para intercambio rápido de información entre microservicios.

Permite conservar los datos almacenados incluso después de reiniciar el contenedor.

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
37   postgres_master:
38     image: postgres:15
39     container_name: c5_postgres_master
40     environment:
41       POSTGRES_USER: c5admin
42       POSTGRES_PASSWORD: c5secret
43       POSTGRES_DB: c5db
44       POSTGRES_REPLICATION_USER: replicator
45       POSTGRES_REPLICATION_PASSWORD: replicasecret
46     volumes:
47       - postgres_master_data:/var/lib/postgresql/data
48       - ./postgres/master-init.sh:/docker-entrypoint-initdb.d/master-init.sh
49     ports:
50       - "5432:5432"
51     networks:
52       - c5net
53     restart: unless-stopped
```

Este servicio corresponde a la base de datos principal donde se realizan las operaciones de escritura.

Estas variables crean automáticamente la base de datos principal y las credenciales de acceso.

Expone PostgreSQL en el puerto estándar 5432.

```

c5_campus-GRPC > docker-compose.yml
55     postgres_replica:
56         image: postgres:15
57         container_name: c5_postgres_replica
58         environment:
59             POSTGRES_USER: c5admin
60             POSTGRES_PASSWORD: c5secret
61             PGPASSWORD: replicasecret
62         volumes:
63             - postgres_replica_data:/var/lib/postgresql/data
64             - ./postgres/replica-init.sh:/docker-entrypoint-initdb.d/replica-init.sh
65         depends_on:
66             - postgres_master
67         ports:
68             - "5433:5432"
69         networks:
70             - c5net
71         restart: unless-stopped

```

Este servicio recibe las alertas iniciales provenientes de los dispositivos del campus.

Las variables de entorno configuran el puerto de ejecución, el identificador de la instancia y las direcciones de Mosquitto y Redis.

El sistema cuenta con tres instancias:

Esto permite distribuir la carga de trabajo y mejorar la disponibilidad del servicio.

```

c5_campus-GRPC > docker-compose.yml
118     nginx:
119         image: nginx:alpine
120         container_name: c5_nginx
121         ports:
122             - "8080:80"
123         volumes:
124             - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
125         depends_on:
126             - reception_1
127             - reception_2
128             - reception_3
129         networks:
130             - c5net
131         restart: unless-stopped

```

Nginx funciona como balanceador de carga para las tres instancias de Reception.
Asegura que todas las instancias estén disponibles antes de iniciar el balanceador.
Expone el servicio a través del puerto 8080.

```
docker-compose.yaml X
c5_campus-GRPC > docker-compose.yaml
133     priority:
134         build: ./services/priority
135         container_name: c5_priority
136         environment:
137             - GRPC_PORT=50051
138             - HTTP_PORT=3003
139             - REDIS_URL=redis://redis:6379
140         volumes:
141             - ./services/priority/rules.json:/app/rules.json
142         depends_on:
143             - redis
144         ports:
145             - "50051:50051"
146             - "3003:3003"
147         networks:
148             - c5net
149         restart: unless-stopped
```

Este microservicio clasifica las alertas según su nivel de gravedad.
Define los puertos utilizados por gRPC y la API REST.
Permite modificar las reglas de clasificación sin reconstruir el contenedor.

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
151     geolocation:
152         build: ./services/geolocation
153         container_name: c5_geolocation
154         environment:
155             - PORT=3002
156             - REDIS_URL=redis://redis:6379
157             - PRIORITY_GRPC=priority:50051
158         depends_on:
159             - redis
160             - priority
161         networks:
162             - c5net
163         restart: unless-stopped
```

Su función es determinar la ubicación geográfica de una alerta

Esta configuración le indica cómo conectarse al servicio Priority mediante gRPC.

Geolocation necesita que Redis y Priority estén activos antes de iniciar.

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
165     notifications:
166         build: ./services/notifications
167         container_name: c5_notifications
168         environment:
169             - PORT=3004
170             - REDIS_URL=redis://redis:6379
171         depends_on:
172             - redis
173         ports:
174             - "3004:3004"
175         networks:
176             - c5net
177         restart: unless-stopped
```

Gestiona el envío de notificaciones relacionadas con las emergencias.

Expone su API mediante el puerto 3004.

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
179   history:
180     build: ./services/history
181     container_name: c5_history
182     environment:
183       - PORT=3005
184       - DB_MASTER=postgres://c5admin:c5secret@postgres_master:5432/c5db
185       - DB_REPLICA=postgres://c5admin:c5secret@postgres_replica:5432/c5db
186       - REDIS_URL=redis://redis:6379
187     depends_on:
188       - postgres_master
189       - postgres_replica
190       - redis
191     ports:
192       - "3005:3005"
193     networks:
194       - c5net
195     restart: unless-stopped
```

Almacena y consulta el historial de eventos del sistema.

Permite conectarse tanto a la base principal como a la réplica para almacenar y consultar información histórica.

Expone su API en el puerto 3005.

```
docker-compose.yml X
c5_campus-GRPC > docker-compose.yml
197 transcription:
198   build: ./services/transcription
199   container_name: c5_transcription
200   environment:
201     - REDIS_URL=redis://redis:6379
202     - CAM_AUDIO_URL=http://192.168.2.23:8080/audio.wav
203   depends_on:
204     - redis
205   extra_hosts:
206     - "host.docker.internal:host-gateway"
207   networks:
208     - c5net
209   restart: unless-stopped
```

Este servicio procesa audio proveniente de cámaras o dispositivos externos.

Especifica la dirección desde donde se obtendrá el archivo de audio para ser procesado.

Permite que el contenedor pueda comunicarse con servicios ejecutándose directamente en la máquina anfitriona.

De esta manera, el archivo docker-compose.yml no solo crea los contenedores, sino que también define toda la arquitectura distribuida del sistema C5 Campus, permitiendo que cada microservicio cumpla una función específica dentro del proceso de atención de emergencias.